



UNIVERSITÉ PARIS SACLAY

FACULTÉ DES SCIENCES

Projet application de Dessin

Présenté par :

Sarr Sophie - Pourchot Alistair

Groupe : LDD3-MAG-INFO

Contents

1	Introduction	2
2	Architecture de l'application	2
2.1	Organisation des fichiers	2
2.2	Hierarchie des formes	2
2.3	La classe Modele	3
2.4	L'enum Action	3
3	Rendu graphique	3
3.1	La ZoneDessin	3
3.2	Poignées de redimensionnement	3
4	Fonctionnalités implémentées	4
4.1	Dessin de formes geometriques	4
4.2	Formes pleines ou creuses	4
4.3	Selection et déplacement	4
4.4	Redimensionnement	4
4.5	Couleur	4
4.6	Ordre Z	4
4.7	Duplication	5
4.8	Sauvegarde et chargement	5
4.9	Export image	5
4.10	Ratio du canvas	5
5	Interface utilisateur	5
6	Tableau récapitulatif des fonctionnalités	6
7	Conclusion	6

Introduction

Ce rapport présente l'implémentation finale de l'application de dessin développée dans le cadre du projet PIIA. L'application a été réalisée en C# avec le framework Windows Forms, dans le namespace TP8. Elle est destinée à des chefs de projet souhaitant créer des visuels à intégrer dans des documents (comme Word ou PowerPoint).

L'ensemble des fonctionnalités obligatoires du cahier des charges a été implémenté, ainsi que la fonctionnalité bonus de sauvegarde d'un fichier. Quatre fonctionnalités supplémentaires ont été ajoutées afin d'améliorer l'expérience utilisateur.

Architecture de l'application

Organisation des fichiers

Le projet est structuré autour de six fichiers sources principaux :

- `FormeGeo.cs` : classe abstraite de base pour toutes les formes.
- `Modele.cs` : modèle de données ; contient aussi les classes concrètes `Rectangle`, `Disque`, `Droite`, `Dessin` et `Texte`.
- `ZoneDessin.cs` : composant graphique héritant de `Control`, responsable du rendu et de la gestion des événements souris.
- `Sérialisation.cs` : sauvegarde et chargement au format JSON.
- `enumAction.cs` : énumération des modes d'interaction.
- `TP8.cs` / `TP8.Designer.cs` : formulaire principal.

Hierarchie des formes

La classe abstraite `FormeGeo` définit l'interface commune à toutes les formes. Elle encapsule la `Position` (de type `Point`), la couleur, un `ZFactor` pour l'ordre d'empilement et un booléen `fill` indiquant si la forme est pleine ou creuse.

Listing 1: Extrait de `FormeGeo.cs`

```
public abstract class FormeGeo
{
    private Point position;
    private bool fill;

    public abstract bool ContientPoint(Point p);
    public abstract Color getCouleur();
    public abstract void setCouleur(Color couleur);
    public int getZFactor() { return ZFactor; }
    public bool getFill() { return fill; }
}
```

Les cinq classes concrètes qui en héritent sont : `Rectangle` (`Largeur`, `Hauteur`), `Disque` (`Rayon`), `Droite` (`PointDebut`, `PointFin`), `Dessin` (polyligne stockée dans une `List<Point>` alimentée par `AjouterPoint()`) et `Texte` (`Contenu`, `Police`).

La classe Modele

Modele maintient deux listes : `formes` (toutes les formes du canvas) et `formesSelectionnees` (la selection courante). Elle expose les opérations fondamentales : `ajouterForme()`, `supprimerForme()`, `setSelection()`, `clearSelection()`, `dupliquerSelection()`, `setCouleurSelection()`, `setZFactorSelectionAvant()` et `setZFactorSelectionArriere()`.

L'enum Action

Les modes d'interaction sont modelisés par l'enumeration `Action` definie dans `enumAction.cs` :

Listing 2: enumAction.cs

```
public enum Action
{
    deplacer, dessiner, creerRectangle,
    creerDisque, creerDroite, selectionner, creerTexte
}
```

A chaque instant, le `Modele` détient l'action en cours via `getAction()` et `setAction()`, ce qui permet à `ZoneDessin` d'adapter le comportement des événements souris en conséquence, via un ensemble de conditions sur la valeur de l'action.

Rendu graphique

La ZoneDessin

`ZoneDessin` hérite de `Control` et active le double-buffering (`DoubleBuffered = true`) pour éviter le scintillement. Le rendu est réparti entre deux méthodes surchargées.

`OnPaintBackground` dessine l'état définitif du canvas. Les formes sont d'abord triées par `ZFactor` croissant via LINQ (`OrderBy(f => f.getZFactor())`), garantissant l'ordre d'empilement correct. Pour chaque forme, `getFill()` détermine si l'on appelle `FillRectangle` / `FillEllipse` (pleine) ou `DrawRectangle` / `DrawEllipse` (creuse). Les formes selectionnées sont ensuite surlignées d'un contour jaune en pointillés.

`OnPaint` superpose les previews en cours de création (rectangles, disques, droites, polygones) avec un canal alpha de 80 pour donner un retour visuel immédiat pendant le glisser-déposer.

Poignées de redimensionnement

La méthode `GetPoignées(FormeGeo f)` calcule et retourne un tableau de petits rectangles de `TAILLE_POIGNEE = 8` pixels, positionnés aux coins et côtés de la forme. Le nombre de poignées varie selon le type : 8 pour un `Rectangle` (coins + milieux), 4 pour un `Disque` (extrémités des axes), 2 pour une `Droite` (`PointDebut` et `PointFin`). Un clic sur une poignée active la variable `poignéeActive` ; chaque `MouseMove` modifie alors la géométrie par delta de déplacement.

Fonctionnalités implémentées

Dessin de formes geometriques

Cinq types de formes peuvent etre dessinés par cliquer-glisser sur la `ZoneDessin`. Pour le `Rectangle`, le coin superieur gauche et les dimensions sont calculés depuis `positionInitiale` et la position courante. Pour le `Disque`, le centre est fixé au premier clic et le rayon est la distance euclidienne jusqu'au relachement. Pour la `Droite`, `PointDebut` est fixé au clic et `PointFin` est mis à jour à chaque `MouseMove`. Le `Dessin` libre est une polyligne construite en appelant `AjouterPoint()` à chaque `MouseMove`. Le `Texte` est placé à la position du clic après saisie via `InputBox`.

Formes pleines ou creuses

Le booléen `fill` est transmis au constructeur de `FormeGeo` lors de la création de la forme. L'utilisateur bascule ce mode depuis la barre d'outils ; la valeur est transmise à `ZoneDessin` via `setFill(bool v)`. A l'affichage, `getFill()` détermine le mode de rendu pour chaque forme indépendamment.

Selection et déplacement

L'outil `selectionner` permet de selectionner une ou plusieurs formes par rectangle de lasso. Les formes dont le bounding box intersecte le rectangle de selection sont ajoutées à `formesSelectionnées` via `IntersectsWith()`. La selection est matérialisée par un rectangle bleu semi-transparent pendant le glisser, puis par un contour jaune en pointillés sur les formes selectionnées.

L'outil `deplacer` permet le glisser-deposer de la selection. Le delta (`deltaX`, `deltaY`) est appliqué à chaque forme ; pour les `Droite` et `Dessin`, chaque point constitutif est translate individuellement.

Redimensionnement

Lorsqu'une seule forme est selectionnée en mode `déplacer`, les poignées calculées par `GetPoignées()` sont affichées. Selon la poignée active (`poignéeActive`), la géometrie de la forme est modifiée différemment : pour un `Rectangle`, les 8 poignées permettent de redimensionner dans toutes les directions avec ajustement simultané de la position et des dimensions ; pour un `Disque`, le rayon est ajusté ; pour une `Droite`, les deux extrémités sont déplaçables indépendamment.

Couleur

La couleur active est stockée dans `ZoneDessin.Couleur` et appliquée à toute nouvelle forme. Pour modifier la couleur d'une forme existante, l'utilisateur la selectionne puis choisit une couleur ; `setCouleurSelection()` itère sur `formesSelectionnées` et appelle `setCouleur()` sur chacune.

Ordre Z

L'ordre d'empilement repose sur l'attribut `ZFactor`. `setZFactorSelectionAvant()` assigne le `ZFactor` maximum existant incrémenté de un ; `setZFactorSelectionArriere()`

assigne le minimum décrementé de un. Le rendu trie les formes par **ZFactor** croissant à chaque rafraîchissement via **OrderBy**, garantissant la cohérence visuelle sans réindexation.

Duplication

`dupliquerSelection()` crée une copie de chaque forme sélectionnée décalée de 10 pixels. Chaque type est traité explicitement afin de recopier tous les attributs spécifiques. Les nouvelles formes deviennent immédiatement la sélection active, permettant de les déplacer directement après duplication.

Sauvegarde et chargement

La classe statique **Serialisation** utilise **System.Text.Json** pour persister l'état du canvas en JSON. Chaque forme est sérialisée avec son type (champ **"type"**), ses coordonnées, sa couleur au format **"R,G,B"**, son booléen **fill** et ses attributs spécifiques. **Charger()** reconstruit les objets **FormeGeo** par un **switch** sur le champ **"type"**, permettant de retrouver exactement l'état du dessin à la réouverture, ce qu'un fichier image classique ne permettrait pas.

Export image

Le canvas peut être exporté en image raster (PNG, JPEG, BMP) via le menu. Le contenu de **ZoneDessin** est rendu dans un **Bitmap** aux dimensions du contrôle par **Graphics.FromImage()**, puis sauvegardé par **Image.Save()** avec le format sélectionné dans une boîte de dialogue **SaveFileDialog**.

Ratio du canvas

L'utilisateur peut choisir le ratio d'affichage parmi cinq valeurs : 1:1, 2:1, 3:2, 4:3 et 16:9. La méthode **resize(int longueur, int largeur)** de **ZoneDessin** redimensionne le contrôle, et **replace(Point p)** le recentre dans la fenêtre par un calcul de position.

Interface utilisateur

La barre d'outils est organisée en groupes fonctionnels séparés par des séparateurs visuels : outils de sélection/action, outils de forme, mode plein/creux, palette de couleurs et ordre Z. Ce regroupement respecte l'heuristique H4 de Nielsen (cohérence et standards). L'outil actif est surligné évisuellement (H1, visibilité du statut). Les icônes accompagnant chaque bouton permettent la reconnaissance immédiate sans mémorisation (H6).

Tableau récapitulatif des fonctionnalités

Fonctionnalité	OK	Type
Dessiner des formes (ligne, rectangle, ellipse, dessin libre, texte)	✓	Obligatoire
Déplacer les formes	✓	Obligatoire
Choisir la couleur des formes dessinées	✓	Obligatoire
Changer la couleur des formes sélectionnées	✓	Obligatoire
Redimensionner les formes (poignées)	✓	Obligatoire
Mettre une forme au premier plan / dernier plan (ZFactor)	✓	Obligatoire
Sauvegarde dans un format personnalisée (JSON)	✓	Bonus
Zoom sur une partie du dessin	×	Bonus
Formes pleines ou creuses (getFill())	✓	Supplémentaire
Duplication d'une forme sélectionnée (dupliquerSelection())	✓	Supplémentaire
Export image (PNG, JPEG, BMP)	✓	Supplémentaire
Choix du ratio du canvas (1:1, 2:1, 3:2, 4:3, 16:9)	✓	Supplémentaire

Conclusion

Presque l'ensemble des fonctionnalités du cahier des charges a été implémenté, ainsi qu'une des deux fonctionnalités bonus du sujet. L'architecture retenue – classe abstraite **FormeGeo**, modèle centralisé dans **Modele**, rendu déporté dans **ZoneDessin** – permet d'étendre facilement l'application avec de nouveaux types de formes sans modifier les composants existants. Quatre fonctionnalités supplémentaires ont été ajoutées, chacune répondant à un besoin concret du cas d'usage identifié lors du rapport préalable.